Enterprise Real-Time Data Engineering

Day 2 Master Training & Practical Lab Guide

High-Throughput Production, Reliable Consumption & S02 Foundations Remediation

Lead Instructor: Mohamed Fouad Fakhruldeen (Fakhr.me)

Course Portal: fakhr.me/KafkaSpark

Code & Labs Repository: github.com/fakhruldeen/KafkaSpark

🎯 Pedagogical Delivery Architecture: The 30 / 20 / 50 Formula

To ensure trainees gain both architectural clarity and deep muscle memory, Day 2 follows a strict **30% Lecture / 20% Demonstration / 50% Hands-On Trainee Practice** structure:

```
┌──────────────────────────────────────────────────────────────────────┐
│ 30% CONCEPTUAL LECTURES (90 Min)                                     │
│ • S02 Bridge: Physical Partitions, MurmurHash2 & ISR Durability (30m)  │
│ • S03 Internals: Dual-Threading, RecordAccumulator & Idempotence (30m) │
│ • S04 Internals: Consumer Groups, Cooperative Rebalance & DLT (30m)  │
├──────────────────────────────────────────────────────────────────────┤
│ 20% INSTRUCTOR LIVE LAB DEMOS (60 Min)                               │
│ • Demo 1: Topic Anatomy & Physical Segment Forensic Dump (20m)       │
│ • Demo 2: Producer Micro-Batching & Snappy Throughput Tuning (20m)   │
│ • Demo 3: Live Split-Screen Consumer Rebalancing & Poison Pill DLT (20m) │
├──────────────────────────────────────────────────────────────────────┤
│ 50% TRAINEE HANDS-ON WORKSTATION PRACTICE (150 Min)                  │
│ • Lab Track 1 (S02): Topic Creation, Key Routing & Chaos Broker Failover │
│ • Lab Track 2 (S03): High-Throughput Producer, Snappy & Pytest Suite   │
│ • Lab Track 3 (S04): Dual-Worker Rebalancing, Crash Test & DLT Isolation │
│ • Lab Track 4 (Capstone): Black Friday Integrated Cluster Resilience Drill │
│ • Verification Gates: verify_s02.sh, verify_s03.sh, verify_s04.sh & Quizzes │
└──────────────────────────────────────────────────────────────────────┘
```

⏱️ Detailed Day 2 Clock Schedule (5 Contact Hours)

Time Slot	Allocation	Focus & Activity	Deliverable / Command
09:00 – 09:30	**30% Lecture**	**S02 Remediation:** Partitions, MurmurHash2 routing, and In-Sync Replicas (ISR)	Whiteboard & Slides S02
09:30 – 10:00	**30% Lecture**	**S03 Deep Dive:** Producer dual-threaded engine, `RecordAccumulator`, and Idempotence	Slides S03
10:00 – 10:30	**30% Lecture**	**S04 Deep Dive:** Consumer groups, Cooperative Sticky rebalance, and DLT pattern	Slides S04
10:30 – 10:50	**20% Demo**	**Instructor Demo 1:** `kafka-topics`, MurmurHash2 verification, and binary `kafka-dump-log.sh`	Terminal (Projector)
10:50 – 11:10	**20% Demo**	**Instructor Demo 2:** Python Producer execution, batching vs latency tuning (`linger.ms`)	`python/s03_producer.py`
11:10 – 11:30	**20% Demo**	**Instructor Demo 3:** Multi-terminal rebalance live and poison-pill routing to DLT	`python/s04_consumer.py`
11:30 – 12:00	*Break*	*Coffee / Networking Break*	—
12:00 – 12:35	**50% Lab**	**Trainee Lab 1 (S02):** Replicated topic creation, key routing proof, chaos broker kill	`./verify_s02.sh` + Quiz
12:35 – 13:15	**50% Lab**	**Trainee Lab 2 (S03):** Producer configuration, Snappy compression & Idempotent Pytest	`./verify_s03.sh` + Quiz
13:15 – 14:00	**50% Lab**	**Trainee Lab 3 (S04):** Dual-terminal consumer group, crash failover, and DLT routing	`./verify_s04.sh` + Quiz
14:00 – 14:30	**50% Lab**	**Trainee Lab 4 (Capstone):** Integrated Black Friday outage simulation drill	End-to-End Cluster Test

📚 Part 1: Conceptual Grounding Lectures (30% – 90 Minutes)

1.1 S02 Remediation: Distributed Partitioning & Durability (30 min)

- **The Core Analogy:** In a supermarket handling 100,000 shoppers per hour, a single cashier queue creates an infinite bottleneck. A topic is split into multiple parallel queues called **Partitions**, spread across distinct server nodes.

- **Deterministic MurmurHash2 Partitioning:**
$$\text{Partition} = \text{toPositive}(\text{MurmurHash2}(\text{keyBytes})) \pmod{\text{numPartitions}}$$
- If `Key = null`: Round-robin / sticky batching evenly distributes traffic.
- If `Key ≠ null`: Identical keys are guaranteed to land on the identical partition index in strict sequential order.
- **Physical Segment Anatomy on Linux Filesystem:**
- `.log`: Raw binary append-only records.
- `.index`: Sparse memory-mapped index translating logical offsets to physical byte offsets on disk.
- `.timeindex`: Maps message timestamps to offsets.
- **The High-Availability Contract (ISR & Quorum):**
- **Leader:** Handles 100% of read and write requests for that partition.
- **Followers:** Continuously fetch records from the leader to stay synchronized.
- **ISR (In-Sync Replicas):** The dynamic set of healthy nodes keeping up with the leader.
- **Durability Gate:** With `acks=all` and `min.insync.replicas=2`, if the active ISR count drops below 2, writes are rejected with `NotEnoughReplicasException`, guaranteeing **zero silent data loss**.

1.2 S03 Producer Internals & Delivery Guarantees (30 min)

- **Dual-Threaded Architecture:**
- **Application (Main) Thread:** Serializes event, computes target partition, allocates/acquires RAM from `BufferPool`, and appends message to the `RecordAccumulator`. Returns immediately (non-blocking).
- **Sender (I/O) Thread:** Asynchronously monitors batches. When `batch.size` (e.g. 32KB) fills up or `linger.ms` (e.g. 10ms) expires, the batch is compressed with **Snappy** and dispatched over socket buffers to the partition leader.
- **Delivery Semantics (`acks`):**
- `acks=0`: Fire-and-forget. Maximum throughput, zero durability.
- `acks=1`: Confirmed once the partition leader writes to local cache/disk. Vulnerable to un-replicated leader crash.
- `acks=all` (-1): Confirmed only when all active ISR replicas acknowledge disk commit.
- **Idempotent Producer (`enable.idempotence=true`):**
- Problem: If a network ACK drops after broker commit, producer retries, creating duplicate transactions.
- Solution: Kafka assigns each producer a unique **Producer ID (PID)** and monotonic **Sequence Numbers** ($0, 1, 2\dots$) per partition. If the broker receives a duplicate sequence number, it discards the record but returns success ACK. Zero duplicates!

1.3 S04 Consumer Groups, Rebalancing & Resiliency (30 min)

- **Consumer Group Parallelism:**
- Consumers coordinate to divide topic partitions among themselves.
- **Fundamental Scalability Law:** A single partition cannot be read by multiple consumers in the same group. If a topic has 3 partitions, a 4th consumer in the same group stays idle.
- **Cooperative Sticky Rebalance Protocol:**
- **Legacy Eager:** Stopped the world; all consumers surrendered all partitions during any group change.
- **Cooperative:** Only revoked/reassigned partitions are migrated; healthy workers continue reading their assigned partitions without latency spikes.
- **Offset Tracking (`__consumer_offsets`):**
- Kafka records the consumer read position as an offset in an internal compact topic.
- **At-Least-Once:** Commit offsets manually *after* business logic finishes (`enable.auto.commit=false`).
- **Poison Pill Isolation (Dead Letter Topic):**
- Badly formatted payloads (e.g., corrupted JSON) will crash consumers repeatedly.
- Pattern: Wrap deserialization in `try/except` , push malformed payloads to `.DLT` , and commit the offset to maintain consumer progress.

💻 Part 2: Instructor Live Demonstrations (20% – 60 Minutes)

Demo 1: Topic Anatomy & Segment Inspection (20 min)

Run from the projector terminal to show trainees real output:

```
# 1. Create a 3-partition, RF=3 topic with durability settings
docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
  --bootstrap-server kafka1:9092 --create --topic demo.inventory \
  --partitions 3 --replication-factor 3 --config min.insync.replicas=2

# 2. Inspect physical files inside container
docker exec -it kafka1 ls -la /var/lib/kafka/data/demo.inventory-0/

# 3. Produce a sample record and dump the binary log
echo "KEY1:{\"item\": \"Widget\", \"price\": 99}" | \
docker exec -i kafka1 /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka1:9092 --topic demo.inventory \
  --property "parse.key=true" --property "key.separator=:"

docker exec -it kafka1 /opt/kafka/bin/kafka-dump-log.sh \
  --files /var/lib/kafka/data/demo.inventory-0/00000000000000000000.log \
  --print-data-log
```

Demo 2: Python Producer Tuning (20 min)

Show students `python/s03_producer.py`:

```
# High throughput run with Snappy compression and Idempotence
python3 -m python.s03_producer --acks all --idempotent --orders 500 --compression snappy
```

Demo 3: Live Split-Screen Cooperative Rebalance (20 min)

Open two terminal splits:
- **Left:** `python3 -m python.s04_consumer --group demo-group --client-id worker-A`
- **Right:** `python3 -m python.s04_consumer --group demo-group --client-id worker-B`

Point out the exact moment partition ownership shifts without service interruption.

🛠️ Part 3: Trainee Hands-On Lab Workstation Practice (50% – 150 Minutes)

> **Terminal Layout for Trainees:**
> Trainees must open 3 terminal tabs in `~/Desktop/Work/Training/Spark/00_Lab`:
> - **Tab 1:** Consumers / Event Monitors
> - **Tab 2:** Producers / Traffic Generators
> - **Tab 3:** Test Suites, Verification Gates & CLI Quizzes

🧪 Trainee Lab 1: S02 Deep Mechanics & Chaos Broker Kill (35 min)

Step 1.1: Stack Health Verification

```
cd ~/Desktop/Work/Training/Spark/00_Lab
tools/check_stack.sh
```

Verify that all 6 services (`kafka1`, `kafka2`, `kafka3`, `schema-registry`, `connect`, `kafka-ui`) return `PASS`.

Step 1.2: Deterministic Key Routing Proof

1. Create topic:
   `bash`
   ```
   docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
     --bootstrap-server kafka1:9092 \
     --create --topic nile.inventory.s02 \
   ```

```
    --partitions 3 --replication-factor 3 \
    --config min.insync.replicas=2
```

2. In **Tab 1 (Consumer)**, listen with partition printing:

```
bash
  docker exec -it kafka1 /opt/kafka/bin/kafka-console-consumer.sh \
    --bootstrap-server kafka1:9092 \
    --topic nile.inventory.s02 \
    --property print.key=true \
    --property print.partition=true \
    --property print.offset=true
```

3. In **Tab 2 (Producer)**, send keyed messages:

```
bash
  docker exec -it kafka1 /opt/kafka/bin/kafka-console-producer.sh \
    --bootstrap-server kafka1:9092 \
    --topic nile.inventory.s02 \
    --property "parse.key=true" \
    --property "key.separator=:"
```

   Paste these records:

```
text
  STORE_CAIRO:{"item": "Laptop", "stock": 10}
  STORE_ALEX:{"item": "Phone", "stock": 25}
  STORE_CAIRO:{"item": "Laptop", "stock": 9}
  STORE_GIZA:{"item": "Tablet", "stock": 5}
  STORE_CAIRO:{"item": "Laptop", "stock": 8}
```

4. **Verification:** Confirm on Tab 1 that all 3 `STORE_CAIRO` messages land on the identical partition number in offset order $0, 1, 2$.

Step 1.3: Chaos Broker Failover & Self-Healing

1. Describe topic to identify the leader for Partition 1:

```
bash
  docker exec -it kafka1 /opt/kafka/bin/kafka-topics.sh \
    --bootstrap-server kafka1:9092 --describe --topic nile.inventory.s02
```

2. Kill Broker 2:

```
bash
  docker stop kafka2
```

3. Re-describe topic: observe active leader reassignment and eviction from ISR.
4. Restart Broker 2:

```
bash
  docker start kafka2 && sleep 5
```

5. Re-describe topic: observe automatic re-synchronization and ISR rejoin.

Step 1.4: Lab 1 Verification Gate & Quiz

```
./verify_s02.sh
python3 quiz.py s02
```

Trainee checkpoint: Must achieve `★ PASS: S02 Lab Requirements 100% Satisfied! ★`.

🧪 Trainee Lab 2: S03 Producer Code, Batching & Idempotence (40 min)

Step 2.1: Run the Production Pipeline

```
python3 -m python.s03_producer --acks all --idempotent --orders 500 --compression snappy
```

Step 2.2: Batching & Performance Benchmark

Compare throughput and latency across configurations:

```
# Unbatched / High Latency
python3 -m python.s03_producer --linger-ms 0 --batch-size 1024 --orders 1000

# High Throughput / Bulk Batched
python3 -m python.s03_producer --linger-ms 25 --batch-size 65536 --orders 1000
```

Note the reduction in execution time and socket round-trips.

Step 2.3: Idempotent Deduplication Pytest Suite

```
.venv/bin/pytest tests/test_s03_producer_logic.py -v
```

Step 2.4: Lab 2 Verification Gate & Quiz

```
./verify_s03.sh
python3 quiz.py s03
```

Trainee checkpoint: Must achieve `★ PASS: S03 Lab Requirements 100% Satisfied! ★`.

🧪 Trainee Lab 3: S04 Consumer Groups, Rebalancing & DLT (45 min)

Step 3.1: Cooperative Rebalance Drill

1. **Tab 1:** Run `python3 -m python.s04_consumer --group nile-workers --client-id worker-A`
2. **Tab 2:** Run `python3 -m python.s04_consumer --group nile-workers --client-id worker-B`
3. Notice partition assignment dynamically balances between workers.
4. In Tab 3, fire 300 orders:
   `bash`
   `python3 -m python.s03_producer --orders 300`
5. Watch both workers process concurrently across partitions.

Step 3.2: Worker Crash & Failover

Kill `worker-A` in Tab 1 with `Ctrl + C`.
Watch Tab 2 (`worker-B`): all 3 partitions are absorbed within seconds without offset loss.

Step 3.3: Poison Pill Isolation to Dead Letter Topic

Inject a corrupted payload:

```
echo "MALFORMED_NON_JSON_CORRUPT_RECORD" | \
docker exec -i kafka1 /opt/kafka/bin/kafka-console-producer.sh \
  --bootstrap-server kafka1:9092 --topic nile.retail.orders
```

Observe consumer safely isolating the poison record to `nile.retail.orders.DLT` and advancing the commit offset.

Step 3.4: Lab 3 Verification Gate & Quiz

```
.venv/bin/pytest tests/test_s04_consumer_logic.py -v
./verify_s04.sh
python3 quiz.py s04
```

Trainee checkpoint: Must achieve `★ PASS: S04 Lab Requirements 100% Satisfied! ★`.

🏆 Trainee Lab 4: Integrated Black Friday Chaos Challenge (30 min)

Enterprise Simulation Exercise:

1. Start 2 consumer workers in `nile-black-friday-group`.
2. Start background traffic generator:
   `bash`
   `python3 -m python.s03_producer --acks all --idempotent --orders 1000 --compression snappy &`
3. During active generation:

- Crash Broker 3: `docker stop kafka3`
- Crash Worker 1: `Ctrl + C`

4. **Validation Checklist:**
- [ ] Did the producer succeed with zero errors? (Yes, $2 \ge \text{min.insync.replicas}=2$).
- [ ] Did the surviving consumer inherit all partitions and complete processing?
- [ ] Did any duplicate records appear? (Zero, verified by Idempotence).

5. Recover cluster: `docker start kafka3`.

🛠️ Student Workstation Troubleshooting Reference

Issue	Root Cause	Immediate Fix
`docker: Error response from daemon: Conflict...`	Previous container session hanging	`docker rm -f $(docker ps -aq)` then `docker compose up -d`
`ModuleNotFoundError: No module named confluent_kafka`	Python venv not active	Use `.venv/bin/python` or execute `source .venv/bin/activate`
`NotEnoughReplicasException`	Live ISR count < `min.insync.replicas`	Run `tools/check_stack.sh` and restart downed brokers (`docker start kafkaX`)
Consumer hangs with no output	Waiting for new records	Add `--from-beginning` flag to read historical log